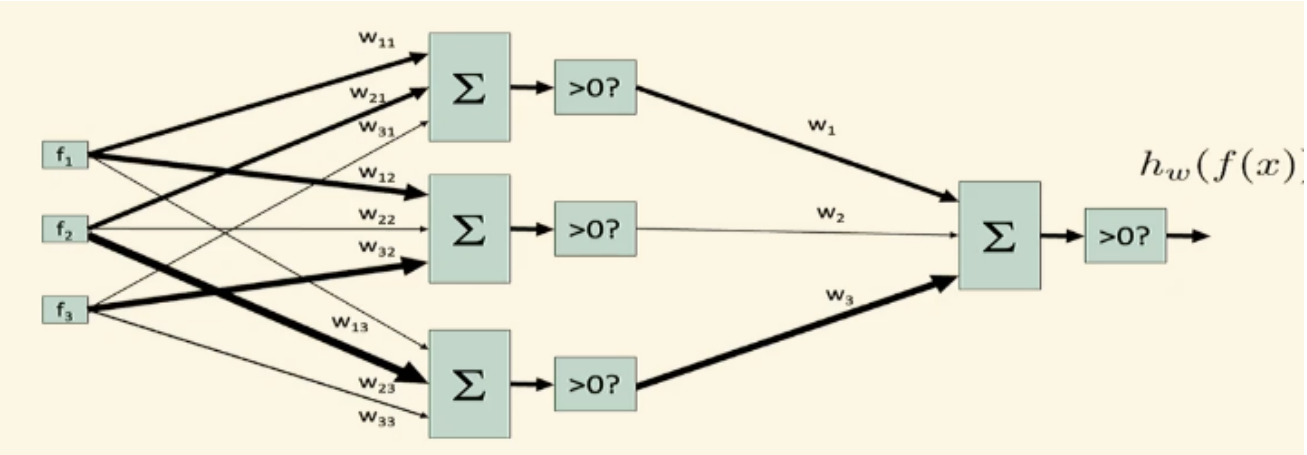
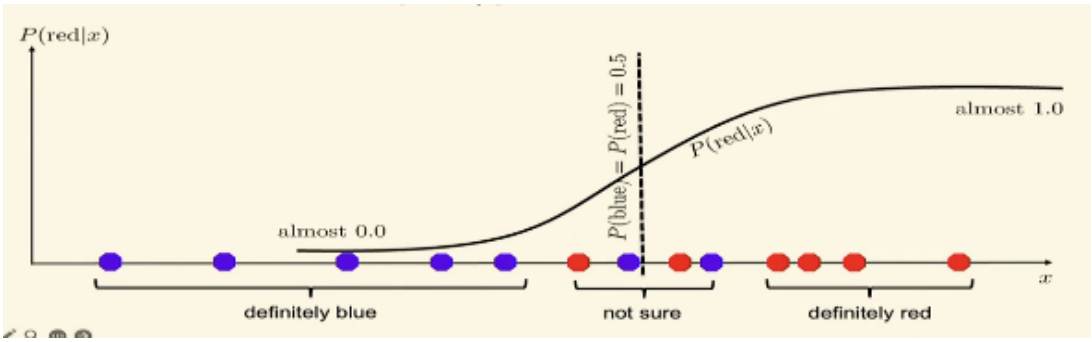
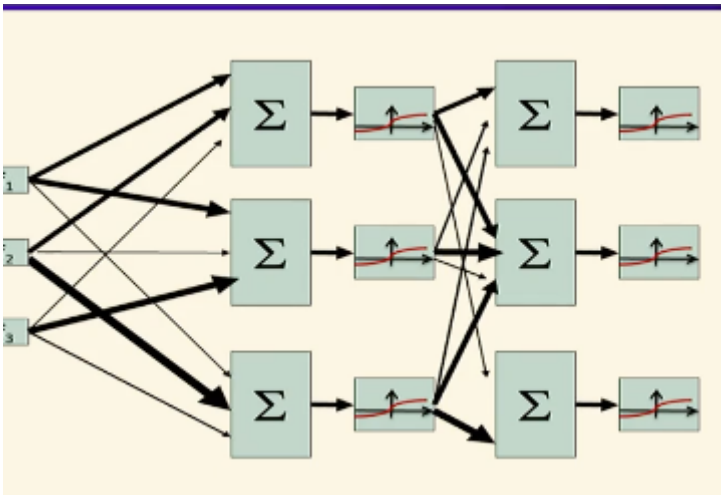
- Cerca di imparare le features dai dati
- 

  - N-Layer perception ha come output >0
  - Per gli N-layer è importante che abbiamo la non-linearità, siccome senò il risultato è troppo semplice
    - La non linearità aumenta la potenzia espessiva
- Praticamente è un percettore concatenati
  - E' un esempio di più layer
  - Il primo layer impara le features
  - E il secondo imparare che classe siamo
  - Idea: I layer più in basso sono cose generiche, e più risali e più è dettagliato
    Si può espandere sempre di più, tanti layer, e siamo arrivati a un certo punto a 150 layer
  - Problema: nessuno dice cosa deve imparare ad ogni layer, quindi un layer potrebbe imparare nulla
  - Ed è impossibile capire cosa cambiare cosa, troppa complessità
  - E come modifichiamo i pesi è più complicato rispetto a prima, essendo che esistono più layer
    - Da qui creiamo un percetron probabilistico
      - Anzichè 0 o 1 abbiamo i floating points
        Perchè questo? Siccome, un cambio da 0 a 1 è un grande cambio, e guardando da fuori la rete neurale se vediamo solo 01
        è difficile comprendere dove avvengono le scelte importanti che ci servono

- Vogliamo cambiare la funzione di attivazione in una softmax, sigmoid, relu

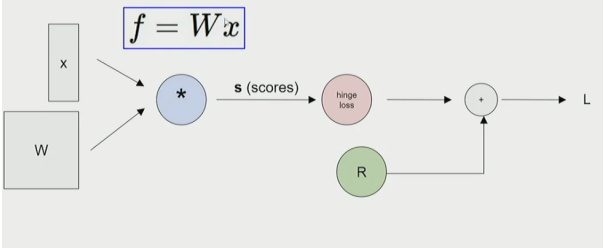


- Aka, siamo passati da una formulazione rigida a una probabilistica
  Così facendo N-Layer cambierà in



  Ora quando abbiamo un errore, possiamo seguire il gradiente (valore) più alto, seguire dove nel layer sta avvenendo l'errore, e propagando il cambiamento indietro
- Prima non potevamo siccome era 1-0 e non si sapeva la differenza
- Si possono così fare più features in 1 singolo perceptor
- In passato per sapere quale migliore si guardava la derivata prima, aka il gradiente
  - Minimizzare l'output dell'ultimo layer cambiando i vari pesi
- Nel presente si fa la differenziazione automatica
  - Utilizzano un **grafo computazionale** che
    - tenendo conto degli input e dei pesi della funzione
    - Vengono messi in una funzione scores
    - Viene poi mandata in una ulteriore funzione chiamata hinge loss
    - E sommando questa hinge loss con +- valore desiderato ottieniamo la nostra loss
    - Questo ci permette di calcolare il valore in avanti, ma anche quello indietro al contrario

- Quello indietro al contrario si chiama back propagation

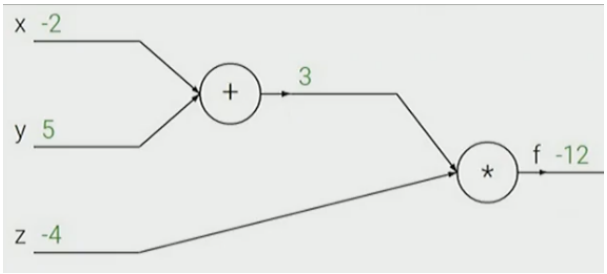


Esempio:

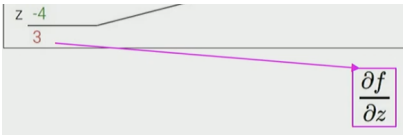
$f(x, y,) = (x + y)$

$f =$

$= x + y$



Si inizia calcolcando $_{ff}((x + y)) = 1$

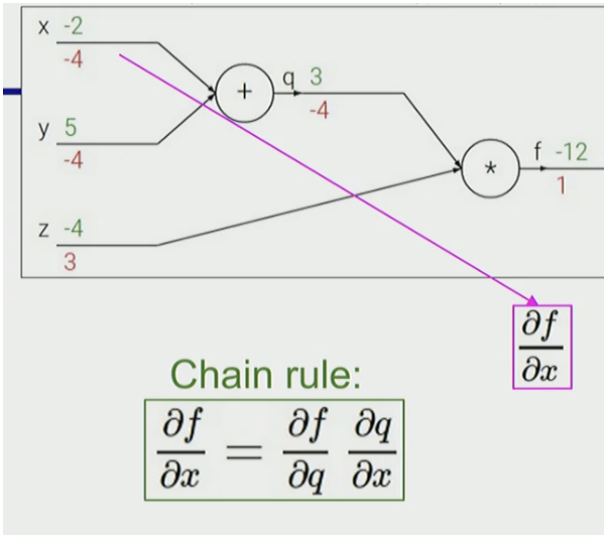


$_f = \ = x + y \quad (-2,) = -2 + \ = 3$

Significato di questo 3:

Se cambio z di 1 valore, allora f cambierà di 3

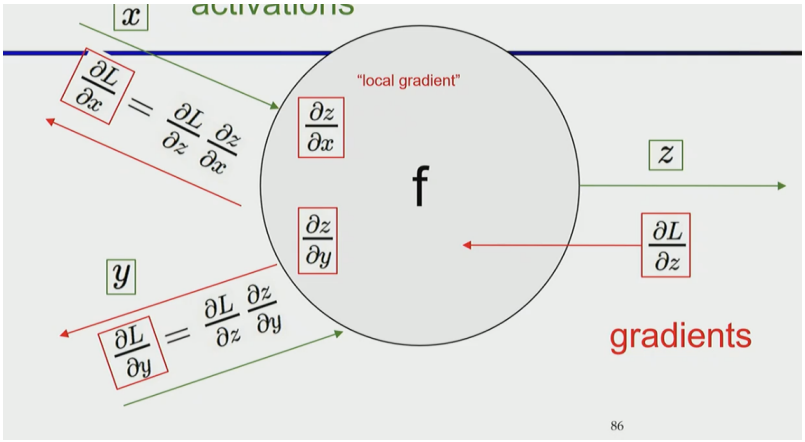
E poi si fà con gli altri, stando attenti però per x e y, siccome li bisogna effettuare la chain rule



Il nostro obiettivo è portare f a 1, ed avendo tutti i valori sappiamo come modificare

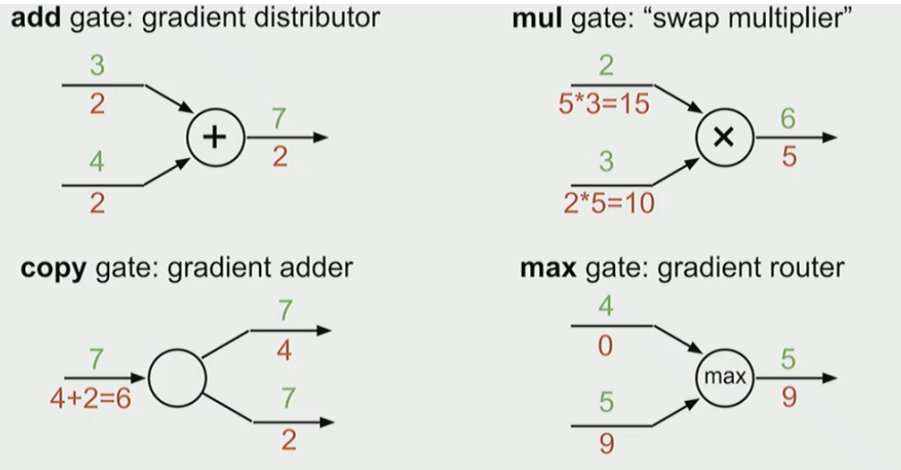
- Quindi, come fanno torch e company per fare la backpropagazione?

- Per ogni percettore assegnano 2 valori:
  - Forward, attivazione (input)
  - Backward, i gradienti
    Che si dividono in:
    - Gradienti locali, quelli riguardante i nostri input
    - 1 gradiente esterno, il gradiente della loss, quello riguardante il nostro output



Inviami indietro la chain rule tra il gradiente loss e il gradiente locale

- RICORDA SEMMPRE: ci deve sempre essere il biass
- Il gradiente per ogni operazione basilare segue delle regole note che danno una mano a velocizzare i calcoli



Aka: per le varie operazioni, sappiamo come il gradiente di output si comporta rispetto a quelli di input

- **SGD** (minibatch stochastic gradiant descent)
  - Quando calcoliamo il gradiente esso dipenderà da qualunque dato del dataset
  - Però se è gigante il dataset, diventa infattibile
  - Ciò che viene fatto è calcolare il gradiente su batch, aka lavorare su un sottoinsieme

Problema:

- Il gradiente non sarà perfetto, è una approssimazione, però si accetta l'approssimazione per diminuire il tempo
- Se il dataset non è omogeneo, potrebbe creare problemi, e potremmo trovarci anche con grandi cambiamenti di pesi e piccolo cambiamento assoluto
- Ed anche che ci potremmo trovare in punti di minimo locale, e non abbiamo abbastanza forza di uscirci
- Il gradiente, in particolare se si scala ecc, potrebbe diventare troppo grande. Quindi si normalizza
  Soluzioni:
- **Momento** (che è anche un parametro che decide quanti gradienti precedenti usare)

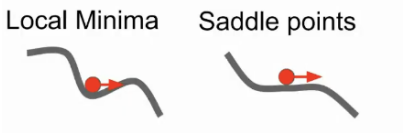
  SGD

  $$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

  SGD+Momentum

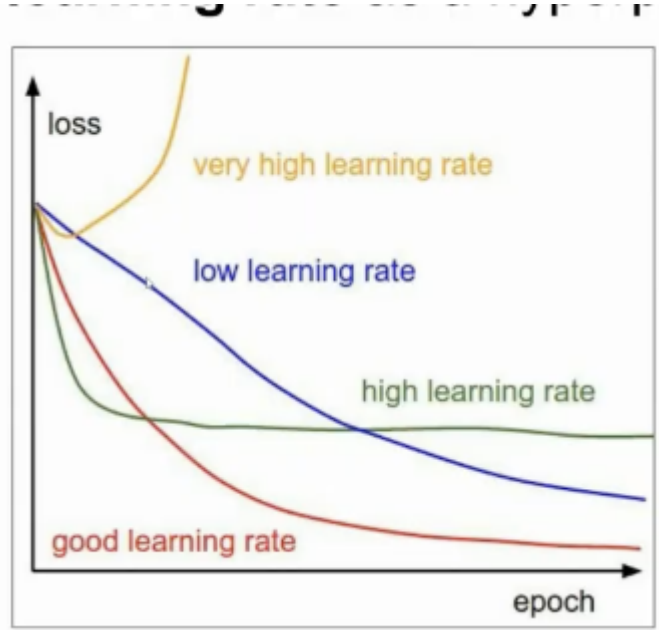
  $$v_{t+1} = \rho v_t + \nabla f(x_t)$$
  $$x_{t+1} = x_t - \alpha v_{t+1}$$

  - Si tiene in considerazione non solo l'ultimo valore del gradiente ma anche la media dei valori precedenti

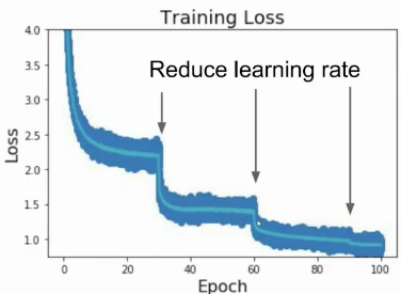
    

    - che scala il gradiente ad ogni passo, che decide di quanto si sposta per ogni passo
    - invece è un numero da 0 a 1 che ci dice quanta importanza dare ai gradienti precedenti
      Frizione
- **AdaGrad**: Si adatta automaticamente il learning rate a seconda del parametro tramite la somma dei quadrati del gradiente, e poi normalizza
- **RMSProp**: risolve il problema di AdaGrad che ci si potrebbe avvicinare a 0 aka rendere il learning rate troppo piccolo
- **Adam**: mette insieme AdaGrad e momento, mantenendo la storia dei gradienti, una media pesata e poi normalizza
  Quello con risultati migliori, ma si suppone siccome i ricercatori hanno ottimizzato molto solo su Adam

- **Iper parametro** molto importante su tutti: Learning rate

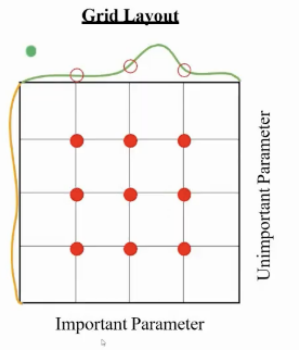


  - Determina i pesi cambian velocemente
  - Low learning rate ci potremmo trovare in un minimo locale dove non usciamo ad uscirci
  - Alto learning rate non riusciamo a convergere nel punto giusto
  - Molto alto learning rate: si potrebbe finire di oscillare troppo dove andiamo solo a crescere
  - Reminder: la loss è il nostro errore
    Uno dei trucchi:
  - Il learning rate decade nel tempo

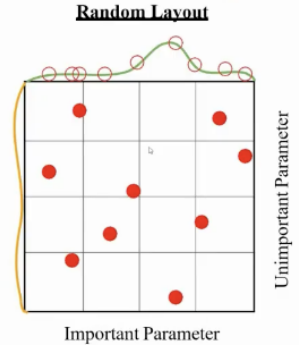


    E qui ci sono vari modi: moltiplica per 0.1 ogni X epoche, oppure diminuizione continua
  - Linear warmup: inizialmente è piccolo, poi sale, e poi decresce
- Come scegliere gli iper-parametri?
  - Si sceglie un sottoinsieme e con poche iterazioni si vede se riesce imparare
  - Poi si gioca sul learning rate affinchè non vadi sui minimi locali

- Poi si scelgono i parametri su un grid layout

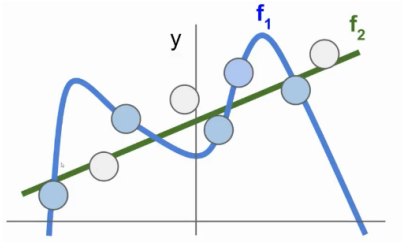


Però notiamo che potremmo mancare i parametri importanti, quindi a volte si va a caso



Nota: noi non sappiamo quali sono i dati importanti, per questo che si va a caso

  - Riassunto cosa si fa:
    - Propagazione in avanti per calcolare le attivazioni (nostri output = input di altri)
    - Calcolano gradienti locali
    - Dopo che arriva alla fine, viene fornito valore riferimento, calcolato l'errore, esso in forma di gradiente globale verrà propagato indietro
    - Attraverso chain rule esso verrà propagato ai precedenti
- Teorema: una rete neurale di 2 layer con un numero sufficente di neuroni può approssima qualunque funzione continua desiderata
  - Nota: troppi neuroni = Overfitting
    Ogni rete multi layer dobbiamo decidere per layer quanti neuroni mettere, le features che vorremmo localizzare
    Ogni layer impara a zone, e poi si fa in un certo senso l'unione
    L'overfitting può succedere anche se ci mettiamo troppe features



Aka non è ottimale cercare di mettere troppe features o troppi neuroni

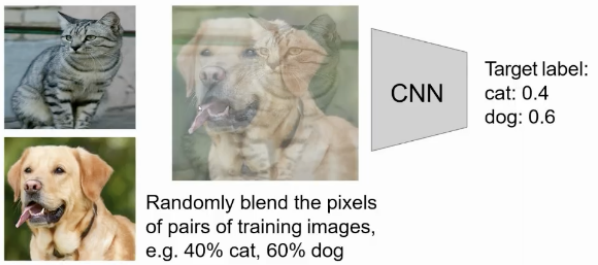
- Se possibile modelli semplici sono migliori (cerchi blue sono training, bianchi testing)
- Esiste anche la **regolarizzazione**, aka un aggiunta nella funzione di loss per evitare che il modello faccia troppo bene
  L'obiettivo è far si che i pesi abbiano valori piccoli, siccome i cambiamenti forti richiedono pesi molto grandi
  Ne esistono tanti, però quello più semplice:
  - Add term to loss (aggiungiamo alla funzione di errore/loss un termine)

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

In common use:

L2 regularization	$R(W) = \sum_k \sum_l W_{k,l}^2$ (Weight decay)
L1 regularization	$R(W) = \sum_k \sum_l \lvert W_{k,l}\rvert$
Elastic net (L1 + L2)	$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + \lvert W_{k,l}\rvert$

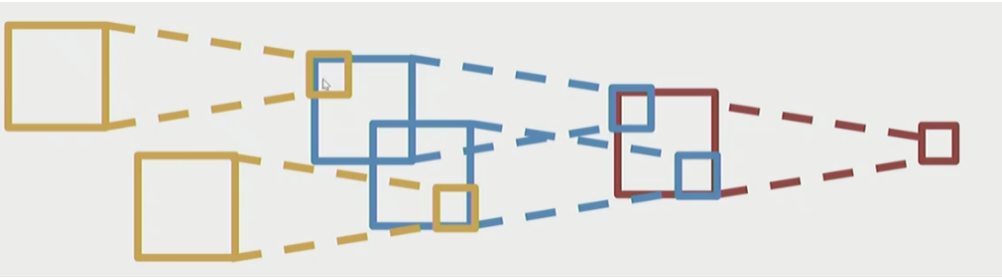
- Se però abbiamo troppi neuroni, c'è l'idea di di uccidere alcuni neuroni durante l'addestramento ogni tanto durante i passi e poi riattivarli subito dopo
  regolarization: **dropout**
  Così facendo i neuroni si devono adattare anche senza certi neuroni
- Regolarizzazione: mixup
  Si combinano 2 tipo di immagini, vengono sommati gli elementi, e poi dare in output una percentuale di quanto cane/gatto ci sono



- **Batch normalization**: si cerca di normalizzare passo per passo i valori prodotti dai layer aventi media 0 e varianza 1
  - in maniera tale che i pesi non devono adattarsi a valori troppo alti/bassi
  - Viene disolito integrato dopo i componenti lineari
  - Idea: neuroni ottengono input con valori stabili

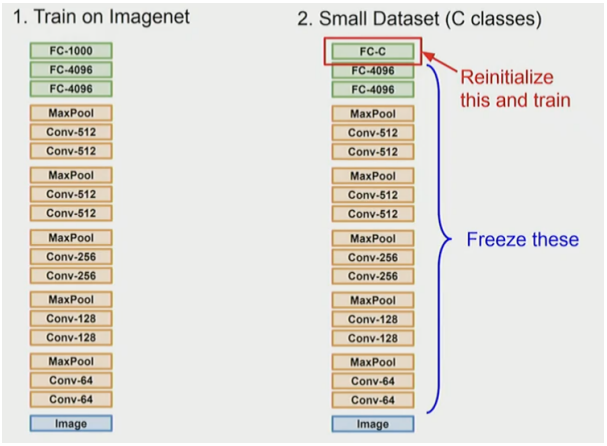
- Nella computer vision c'è un modo per visualizzare le immagini chiamato histogram of oriented gradients

- Divide l'immagine in sottoparti e somma le direzioni più importanti
- E' buono per direzione e movimento
- Problema: è molto grande la grandezza che si creerebbe usandolo
  Per risolvere il problema, si usa il bag of words:
  - Si trasformano le caratteristiche in gruppi che fosse possibile clasterizzare
  - E poi dopo si trovano elementi che lo caratterizzano, e si contano quanti elementi di un certo tipo stanno presenti nell'immagine
    Però è meglio comunque usare il deep learning
  - Problema: le immagini sono molto grandi
    e per questo motivo, sonos tate create le reti neurali covulotive
  - Sfruttano spaziale dell'immagine
  - Problema: richiedono veramente tante immagini annotate
  - Diminuire il numero di pesi
  - Qui vengono costruiti i tipi di layer andando a guardare solo 1 sottoparte
    - Si usano meno pesi
  - Viene creato un filtro con una determinata dimensione
    - Poi viene presa una sottoparte dell'immagine con stessa dimensione del filtro
      Questo però crea un problema: se vogliamo vedere un cane sopra un gatto dentro l'immagine, però sono in gruppi diversi, è un problema
      Siccome insieme nello stesso gruppo darebbero un risultato diverso rispetto a diversi gruppi
      Quindi ciò che viene fatto è che parti dell'immagine di output dipende da parti di altri immagini di input: **receptive fields**

      Per via di questo, anche se i filtri diminuiscono la dimensione dell'immagine, questo la aumenta
      - Per mantenere le diverse caratteristiche le espande in maniera spaziale anzichè di infrmazioni
    - Si fa il prodotto della matrice, e questo output riassumerà le informazioni locali dell'immagine a seconda del filtro
    - Poi passiamo il filtro affianco, e continuamo finchè non abbiamo fatto tutta l'immagine
      - I filtri devono avere dimensioni compatibili

        - Il filtro renderà il prossimo output più piccolo, siccome il filtro riduce
          Continuamo così finchè non siamo alla fine
        - Nota: abbiamo mantenuto la struttura spaziale della rete, siccome anche se abbiamo diviso, ogni elemento è calcolato con i punti vicini dell'immagine di prima
        - Poi c'è il pooling, che cerca di diminuire la quantità di dimensione di immagini abbiamo dentro la rete, che seleziona tra 4 pixel vicini media/massimo/simile
            - Vuole ridurre la dimensionalità di input
            - 4 pixel = Stride, a volte è 3
            - Viene introdotto anche il padding per cercare di conformare la stessa dimensione tra vari layer o per arrotondare le dimensioni del pooling (pixel con 0)
              Nota: se abbiamo 3 canali, dobbiamo avere 1 filtro per canale
- Un altro problema: vengono richieste tante ma tante immagini e richiede molto molto tempo
  Soluzione: transfer learning
    - Le features dei livelli più bassi sono dipendenti dall'immagine di input, che estraggono features riguardante strutture locali
    - Mentre i livelli più alti rispondo al task che stiamo cercando di costruire
    - I livelli più bassi invece più dall'input
      Questo cosa vuol dire: che possiamo riusare i primi layers siccome sono molto generalistici
        - I primi 30 layer di 90 possono essere usati sia per gatti e cani ma anche per mucche e pecore
          Quindi ciò che si fa è freezare i pesi dei primi tot layer così da poterlo riusare

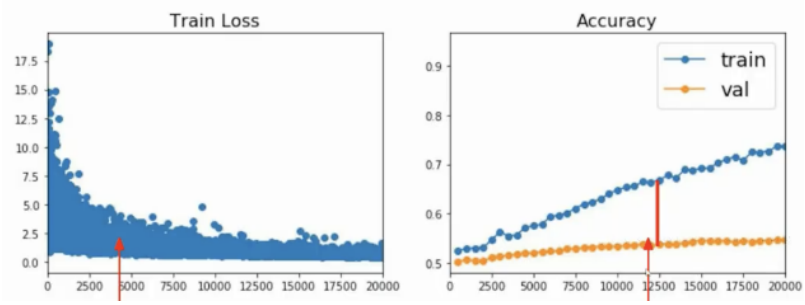


    - Transfer learning = Riutilizzare pesi di modelli già usati
      Questo permette di risparmiare molto tempo e denaro
        - Si allena un dataset molto grande per fare dei buoni primi layer (o si prende online)

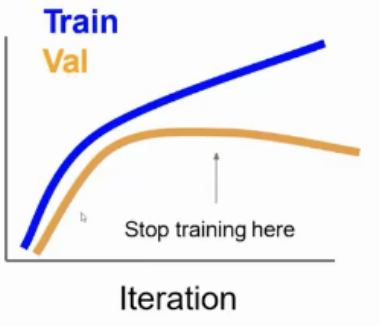
    - E poi si riallenano i layer più in cima con un dataset più piccolo (così si risparmiano immagini)
    - Spesso non si addestra mai una rete da 0
      Ed a seconda di quanti dati abbiamo abbiamo 4 casi:

	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

- Training and testing set errori
  - L'errore del training test va sempre a diminuire più epoche facciamo
  - Però l'accuracia (training/val) dopo un certo punto, il testing/validation test diventerà costante

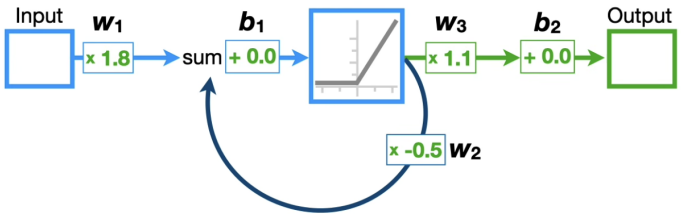


    - Questo vuol dire che il modello si sta concentrando su l'errore
    - E il validation set potrebbe persino diminuire dopo tante epoche

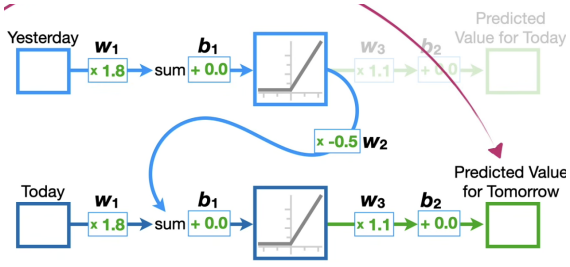


      Per evitare, si allina in batch, un pò sul training set, e poi si valida, e se la validazione inizia a diminuire drasticamente, allora si smette
    - E si fanno tanti snapshot del modello, quando ci fermiamo prendere la snapshot con validation set migliore
  - Non sempre training-test set funziona, per questo si fa la **cross validation**
    - Si divide il tutto un k parti, e poi per k volte si fa k-1 training e 1 validation su 1 parte

        - Nota: la parte di validazione viene fatta ruotare. Questo vuol dire, se prima utilizzo 1 parte per validare, dopo uso 2, poi 3, e così via
- **Modello ensambles**
    - Alleni più modelli contemporaneamente
    - E poi alla fine prende il risultato medio
        - Mischiare un risultato classificatore è complicato
- Come inizializziamo i pesi?
    - Tutti 0.5, però potrebbe creare problemi con la backpropagation ed avere tutti numeri uguali
        - quindi non si fa siccome per tanto tempo non ci sarà apprendimento
    - Numero randomico
- RNN (recurrent neural network)
    - E' una rete neurale che elabora una **sequenza** di dati grazie a connessioni ricorrenti
        - Normalmente le reti neurali sono grafi aciclici
        - Qui noi però abbiamo dei cicli logici
        - Serve per modellare **dipendenze temporali**
            - Però faticano a ricordare dipendenze nel lungo termine siccome il gradiente tende a esplodere/scomparire
    - Es: abbiamo 10 dati per azienda A, 5 dati per azienda B, e vogliamo usare la stessa rete con input diversi
        - Abbiamo diverse tipologie di input alue

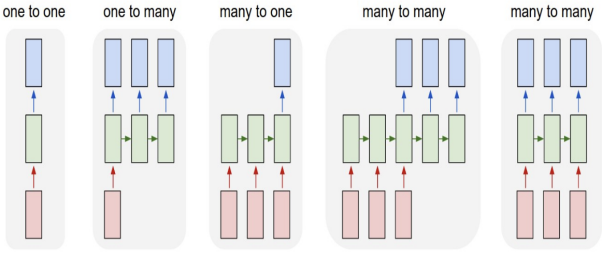


        - Il feedback loop è la linea blue/nera con w2
          All'inizio w2 moltiplicherà y1=0
          Poi quando li passiamo il 2 dato, y1 avrà un valore, che viene moltiplicato per peso w2
          e sommato con nuovo input*w1+b1
          Si può visualizzare meglio così



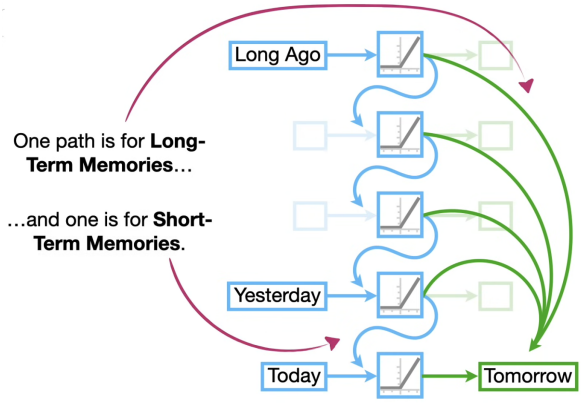
        - Nota: il percettore è lo stesso, nota come i pesi sono gli stessi

    - Questo è l'**unrolling**: la stessa rete neurale ripetuta nel tempo
- Nota: noi teniamo solo 1 stato dentro
    - Questo stato nascosto è la nostra memoria
      Anche se è ricorrente, gli input precedenti li dimentichiamo dopo
    - Ci ricordiamo solo lo stato di quello prima, aka non abbiamo nulla di esplicito
    - Detto questo, li potremmo dare più input che poi concateniamo, le RNN sono molto flessibili
      E ci ricordiamo solo lo stato di ciò che avevamo prima
    - Spesso, siccome possiamo volerli continuare per tanto tempo e magari darli in pasto anche se stessi
      Si usa un termine speciale che, quando lo da in output è finito ciò che voleva dire
- Esistono varie tipologie di RNN a seconda degli input ed output:
    - 1 input => 1 output
      Rete neurale standard
    - Tanti input => 1 output
      Legge 1 sequenza e da 1 output, di solito per classificare una frase
    - 1 input => tanti output
      La rete espande un informazione iniziale, es generare testo da un immagine
    - Molti => molti (con RNN allineate)
      Ogni input ha un output corrispondente
    - Molti => molti (non allineati)
      Non per forza sono allineati, es. traduzione

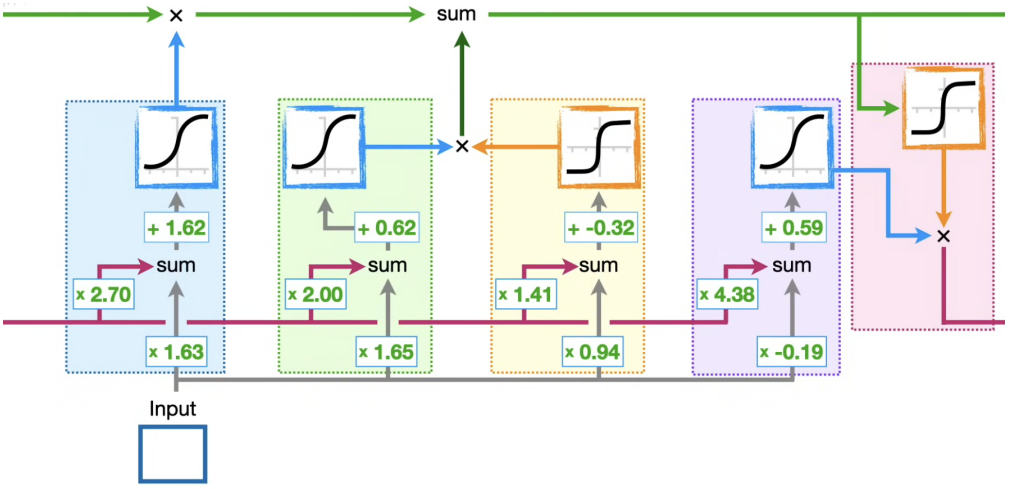


- **Problema**:
    - Più connettiamo il percettore a se stesso, e più diventerà difficile da allenare (esponenziale)
        - Problema del gradiente esplosivo/vanishing
        - Più lo colleghiamo con se stesso, più il gradiente o diventa grande o tende a 0
            - Se il peso è maggiore di 1, allora il gradiente diventerà sempre e sempre più grande
            - Se il peso è minore di 1, allora il gradiente diventerà sempre e sempre più piccolo
              Siccome noi faremo $\text{numero}^{\text{uantevolteabbiamoattolaricorsione}}$

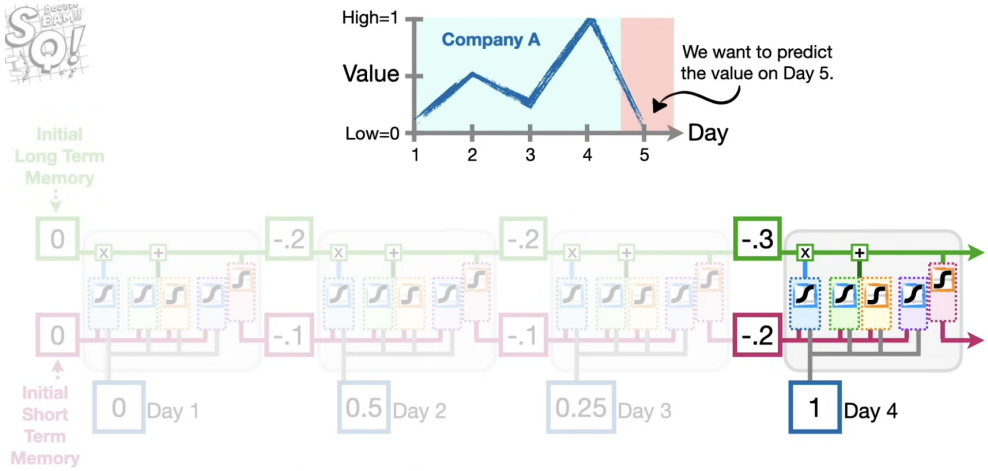
        - Siccome il gradiente tende ad esplodere, e diventare sempre più grande, tendono quindi a fare salti grandi
          Che quest promuove il dimenticare
        - Più grande gradiente, e più i pesi possono cambiare
          Il gradiente tende a diventare sempre e sempre più grande, che questo porterà ad una difficile convergenza
          Step molto grandi
- LSTM
    - E' una RNN che però riduce di molto il exploding/vanishing problem



        - Aggiunta: una linea che per ogni output della sequenza va nell'output finale
        - E' molto complicata per questo motivo
        - Usa **sempre** 2 activation function
            - Sigmoid (ci da un valore tra 0 e 1)
            - Than (ci da un valore tra -1 e 1)
    - Pro:
        - Il gradiente influisce nelle cell state senza moltiplicazioni continue
    - Una sua unità è formata da 3 componenti/fasi:
        1. Fase di dimenticazione (forget gate, il quadrato blue)
        2. Fase di quanto dovremo ricordare (input gate, il quadrato verde ed arancione)
        3. Fase dello short-term memory (output gate, il quadrato viole e rosa)



- Poi replichiamo l'unità N volte quanto la nostra memoria



And the **LSTM** does the math, again, using the exact same **Weights** and **Biases**...

-

Reminder: utilizza stessi bias e pesi in più LSTM è per poter processare più data sequences di lunghezza diversa